

Problem 1

The Trinity Curriculum Development Committee (TCDC) recently submitted a new curriculum, which was approved by the Arts & Sciences Council. The new curriculum requires a student to take a number of classes r_i from m different categories G_i in order to graduate. Let n be the number of courses.

For example, suppose there are $n = 5$ courses A, B, C, D, E and $m = 2$ categories:

- You must take at least $r_1 = 2$ courses from the subset $G_1 = \{A, B, C\}$.
- You must take at least $r_2 = 2$ courses from the subset $G_2 = \{C, D, E\}$.

Then a student who has only taken courses B, C, D cannot graduate, but a student who has taken either A, B, C, D or B, C, D, E can graduate.

Describe and analyze an algorithm to determine whether a given student can graduate. The input to your algorithm is the list of the m categories (each specifying a subset G_i of the n courses and the number of courses r_i that must be taken from that subset) and the list of courses that the student has taken. Justify the correctness of your reduction and analyze the runtime.

Solution: Construct a bipartite graph $G = (V, E)$ where,

$$\begin{aligned}
 V &= M \cup O \cup \{t\} \cup \{s\} \\
 E_c &= \{(i, j) \mid i \in M, j \in O, j \in G_i\} \\
 E_t &= \{(j, t) \mid j \in O, t \in \{t\}\} \\
 E_s &= \{(s, i) \mid i \in M, s \in \{s\}\} \\
 E &= E_c \cup E_t \cup E_s
 \end{aligned}$$

Where M is the set containing categories G_i such that the union of all categories is M , O contains is the set of all courses the student has taken. Define the edges in E_c , E_t with capacity 1 and the capacity of all edges in E_s with r_i corresponding to G_i for $i \in 1 \dots m$. We can now reduce this to computing edge disjoint paths problem, specifically We begin with 0 flow all all edges and run Ford-Fulkerson to compute the max flow from s to vertex t in the constructed graph G . If the resulting max-flow $|f| = \sum_{i=1}^m r_i$, then return true, o.w. false.

Runtime: Constructing the graph takes $O(V + E) = O(MN)$. Running F-F $O(E|f|) = O((MN)N)$ Hence, the total runtime is $O(MN^2)$. Here, N is max number of courses a student can take (all of them).

Correctness: By construction of G , each path to t corresponds to a course requirement fulfillment with the additional course category constraint. Therefore, computing the max flow gives us the number of fulfillment a student has. Therefore, if $|f| = \sum_{i=1}^m r_i$, then the student has fulfilled all of their requirements.

Problem 2

Suppose we are given an array $A[1..m][1..n]$ of non-negative numbers. We want to round A to an integer matrix, by replacing each entry x in A with either $\lfloor x \rfloor$ or $\lceil x \rceil$, without changing the sum of entries in any row or column of A . For example:

$$\begin{bmatrix} 1.2 & 3.4 & 2.4 \\ 3.9 & 4.0 & 2.1 \\ 7.9 & 1.6 & 0.5 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 4 & 2 \\ 4 & 4 & 2 \\ 8 & 1 & 1 \end{bmatrix}$$

1. Describe and analyze an efficient reduction-based algorithm that either rounds A in this fashion, or reports correctly that no such rounding is possible. Justify that your algorithm is correct, and analyze the runtime. [Hint: There is a correct reduction involving a flow network with one vertex per row and one vertex per column.]
2. Justify that a matrix can be rounded if and only if the sum of entries of each column is an integer and the sum of entries of each row is an integer. [Hint: Consider a particular fractional flow on your graph. Recall properties of flow networks with integer capacities.]

Solution:

1. Let set $R = \{r_1, \dots, r_m\}$ and set $C = \{c_1, \dots, c_n\}$ denote the rows and columns of matrix A , respectively. We construct a flow network $G = (V, E)$:

$$V = \{s\} \cup \{t\} \cup R \cup C,$$

$$E = \{s \rightarrow r_x \mid r_x \in R\} \cup \{c_y \rightarrow t \mid c_y \in C\} \cup \{r_x \rightarrow c_y \mid r_x \in R, c_y \in C\}$$

Let the edges connecting sets r_x and c_y all have capacity $\lceil A[x][y] \rceil - \lfloor A[x][y] \rfloor$, the edge (s, r_i) for all edges from $\{s\}$ to R have the capacity equal to sum of fractional parts in row i , and edge (c_j, t) for all edges from C to t with capacity equal to sum of fractional parts in column j . Computing the max flow with Ford-Fulkerson tell us if rounding is possible, i.e. If max flow $|f|$ equals the sum of all fractional parts in A , we can round. Otherwise, it's not possible. To obtain our rounded matrix, we check the following condition

For each $A[i][j]$, if (r_i, c_j) carries a flow, round up to $\lceil A[i][j] \rceil$. O.w. round down to $\lfloor A[i][j] \rfloor$.

Correctness: See that the edges coming out of s and edges going into t allows the graph to satisfies the constraint that the sum must still be the same, since this is essentially taking into account how many entires we need to floor vs. ceil. This is because, each unit of flow represents rounding up an entry and flow conservation ensures that the row and column sums remain unchanged. And if $|f| = \text{sum of fractional parts}$ means all fractional parts are accounted for. If the edge (r_i, c_j) has flow, then it is taken into account into a sum of a fractional part. Thereby, we obtain a rounded matrix if a solution is possible.

Runtime: Constructing the graph takes $O(mn)$ time. See that for $|f|$, $|f| \leq \min(m, n) \cdot \max \text{ fractional part} \leq mn$, Thus, running Ford-Fulkerson algorithm takes $O(mn \cdot mn)$ time. Constructing the rounded matrix takes $O(mn)$. Hence, the total runtime is $O(m^2n^2)$.

2.

1) ($=>$) If matrix A can be rounded, then replacing each fractional part of $A[x][y]$ with either 0

or 1 will not change the sum of the entries in any row or column. Since 0 and 1 are integers, the sum of the integers in each row and column must be an integer.

2) ($\leq$) If the sum of the entries in each row and column is an integer, we can construct an integer flow network as described in 1. The rounding problem is then converted to finding maximum flow in this network. The flow properties ensure that we can round the matrix entries while maintaining the row and column sums.

Problem 3

A Hamiltonian cycle is a (simple) cycle that visits all vertices in the graph exactly once. In Monday's lecture, we will see that DIRHAMCYCLE, detecting whether a given directed graph has a Hamiltonian cycle, is NP-hard. The UNDIRHAMCYCLE problem is the same except that the given graph and cycle to detect are undirected.

Describe a polynomial-time reduction from DIRHAMCYCLE to UNDIRHAMCYCLE, and justify its correctness in order to conclude that UNDIRHAMCYCLE is NP-hard. [Hint: Consider replacing each node u with multiple copies of u . How many copies should be made, how should the copies of one node be connected via edges, and how should the copies of different nodes be connected based on the original edges?]

Solution: We reduce DirHamCycle to UndirHamCycle. Suppose directed graph G is an input graph to DirHamCycle. Observe that we can't directly turn the directed graph into an undirected one by replacing all the edges because it may lead to a Hamiltonian cycle in the undirected graph which don't correspond to valid cycles in the directed graph G . Hence, we construct $G' = (V', E')$, where for each $v \in V$ we add an additional 2 vertices v^-, v^+ and undirected edges (v, v^-) , (v, v^+) , (v^+, u^-) for $vu \in E$, i.e. every directed edge in G is represented by undirected edge from v^+ to u^- in G' . This is a polytime reduction since we are just adding a constant number of vertices and edges.

Correctness: There is a undirected Hamiltonian cycle in G' iff there is a directed Hamiltonian cycle in G

$\rightarrow$: Let C be a Hamiltonian cycle in G . For each vertex v in C , we can construct a path $v^- \rightarrow v \rightarrow v^+$ in G' . The edges between v^+ and the next vertex's u^- in C exist in G' by construction. Therefore, we can construct a Hamiltonian cycle in G' that visits all vertices.

$\leftarrow$: Let C' be a Hamiltonian cycle in G' . Due to the construction of G' , C' must visit v^- , v , and v^+ for each original vertex v . The edges between v^+ and u^- in C' correspond to directed edges (v, u) in G . By removing v^- and v^+ vertices from C' , we obtain a valid Hamiltonian cycle in G .

Problem 4

In this problem we prove that, as for many NP-hard problems, allowing constant additive error does not make the problem any "easier." We define the ALMOSTMAXINDSET problem as follows: Given an undirected graph $G = (V, E)$, output an integer k in the range $|S^*| - 330 \leq k \leq |S^*|$, where $S^* \subseteq V$ is a maximum independent set in G . Prove that ALMOSTMAXINDSET is NP-hard. Here are some hints:

- There is nothing special about the constant 330. Assume it is, say, 3 instead when thinking about the problem.
- Reduce from MAXINDSET and modify the graph in such a way that the size of the maximum independent set in the resulting graph increases in a controlled, predictable way.
- An **incorrect** reduction is to simply add many isolated vertices (e.g., ~ 330) to the graph.

Why is this incorrect? What can you add a sufficiently large amount of instead to obtain a correct reduction?

- The size of any independent set is an integer.

Solution: Note the definition of independent set states that within the set, we can't have adjacent vertices. And given our upperbound for k is still $|S^*|$ Hence, we'll add cliques to our graph in the reduction.

We reduce maximum independent set to almost max independent set. Suppose $G = (V, E)$ of is input to MaxIndSet. Construct a new graph $G' = (V', E')$. And let $V' = V \cup C$, where C is a set of 331 new vertices $E' = E \cup \{(u, v) \mid u, v \in C, u \neq v\} \cup \{(u, v) \mid u \in V, v \in C\}$. This adds a clique of 331 vertices fully connected to all original vertices.

See that the upper bound holds since the maximum independent set S'^* in G' will consist of either the maximum independent set S^* from G , or single vertex from C . Hence we have $|S'^*| = \max(|S^*|, 1)$

Correctness $\rightarrow$: If there MaxIndSet outputs k for G then in G' , we can select this independent set of size k since the 331-clique in G' can contribute at most 1 vertex to any independent set. Therefore, $|S'^*| = \max(k, 1) = k$ (since $k \geq 1$ for any non-empty graph). And we satisfy the AlmostMaxIndSet condition: $|S'^*| - 330 \leq k \leq |S'^*|$, where $|S'^*| = k$

$\leftarrow$: If ALMOSTMAXINDSET outputs k for G' , then:

$$|S'^*| - 330 \leq k \leq |S'^*|$$

$$\max(|S^*|, 1) - 330 \leq k \leq \max(|S^*|, 1)$$

Since $|S^*| \geq 1$ for any graph, it follows that $|S^*| - 330 \leq k \leq |S^*|$.